AstraClean-RAG: Integrating Domain Knowledge and Correction Log for Tabular Data Cleaning

Bingxiang Chen[1 [0009−0002−1512−3612]] and Toni Taipalus[1 [0000−0003−4060−3431]]

Tampere University, Tampere, Finland
{bingxiang.chen, toni.taipalus}@tuni.fi

Abstract. AstraClean-RAG is a human-in-the-loop framework for tabular data cleaning that combines two distinct external memory sources: a *Domain Knowledge Base* (DKB) encoding stable domain constraints and a *Correction Log* (CL) capturing validated historical repairs. This study extends our earlier proof-of-concept by formalizing the complementary roles of these sources through a design inspired by rule-based and case-based reasoning and by evaluating the approach at a broader scale. The evaluation considers multiple datasets, selected dirty columns, and conflict scenarios in which the two evidence sources provide different repair signals. The DKB provides rule-like support for consistency and canonicalization, while the CL provides case-like support for reuse of recurring correction patterns. Results suggest that treating the two sources separately improves reliability over LLM-only and single-source baselines. When the sources disagree, the framework surfaces the disagreement explicitly for human review rather than resolving it automatically. Overall, the findings suggest that the two sources provide complementary support for tabular data cleaning.

Keywords: Data cleaning · Retrieval-augmented generation · Human-in-the-loop · Large language models

1 Introduction

Data cleaning remains one of the most labor-intensive and error-prone stages in modern data preparation workflows [27]. Industry studies consistently show that practitioners invest substantial effort in cleaning before downstream analysis or modeling can begin [7], and recent work further shows that automated cleaning can improve downstream predictive performance [16]. Effective cleaning requires not only identifying potential errors but also the application of domain-specific corrections that depend on both formal constraints and contextual knowledge [8]. For decades, the dominant paradigm has been rule-based, relying on explicit constraints, integrity rules, and dependency specifications [27]. Prior work on data profiling has studied how metadata, dependencies, and constraints can be discovered from relational data [3]. These methods support consistency and transparency, but they still impose authoring, validation, and maintenance burdens. As real-world data evolves, new exceptions emerge, conventions shift, and maintaining rule bases becomes increasingly expensive.

Recent advances in large language models (LLMs) have created new opportunities for data cleaning. LLMs can generate context-aware repair suggestions without task-specific training [29]. However, direct application of LLMs in this setting raises several challenges. First, the same logical error may receive different repairs across runs, undermining consistency. Second, LLM outputs may be weakly grounded in domain-specific constraints and can produce hallucinated values [14]. Third, LLMs have no native mechanism to accumulate and reuse institutional memory across cleaning sessions, meaning that human corrections are not systematically preserved.

Retrieval-augmented generation (RAG) addresses some of these limitations by grounding LLM outputs in retrieved evidence [19]. In data cleaning, existing retrieval-based approaches rely on a user-provided data lake to retrieve similar tuples for repair generation [10], and do not explicitly model stable domain constraints or historical correction practice as separate knowledge sources. This may be limiting in settings where a large-scale reference corpus is unavailable. By contrast, `AstraClean-RAG` separates domain knowledge from correction history into two distinct sources. The DKB can be initialized from a small stratified sample of relatively clean rows, or constructed manually by domain experts. The Correction Log requires no pre-existing data: it starts empty and grows incrementally as human-validated corrections accumulate during cleaning sessions.

This study reports the second-stage development of `AstraClean-RAG`. An earlier proof-of-concept prototype, preserved as version 1.0 in the project repository, introduced the basic idea of combining domain knowledge and correction history for LLM-assisted tabular data cleaning and provided a small-scale feasibility study [5]. The present version advances that prototype in two substantive ways. First, it makes explicit the complementary roles of rule-like domain knowledge and case-like correction history through a dual-source retrieval design inspired by rule-based and case-based reasoning (RBR/CBR). Second, it broadens the empirical evidence through a larger evaluation across multiple datasets and cleaning scenarios. In this study, we address the following research question:

> **RQ:** How does dual-source retrieval, combining stable domain knowledge and reusable historical corrections, affect the reliability of LLM-assisted data cleaning?

The main contributions of this study are: *(1)* an RBR/CBR-inspired conceptualization of dual-source retrieval for tabular data cleaning, where the DKB acts as a rule-like source of stable constraints and the Correction Log (CL) acts as a case-like source of validated historical corrections, *(2)* an extended `AstraClean-RAG` design that preserves and reuses correction history as an evolving external memory, and *(3)* a broader empirical evaluation than in the initial proof-of-concept, covering multiple datasets, columns, and conflict scenarios. The proof-of-concept implementation and evaluation code are available in our public repository.[1]

[1] `https://github.com/bingxiangch/AstraClean-RAG`

The rest of the study is organized as follows. Section 2 reviews related work, and Section 3 describes the system design, followed by the empirical evaluation in Section 4. Section 5 discusses the practical implications and limitations of the study. Section 6 concludes the study.

2 Related Work

2.1 Earlier Approaches to Data Cleaning

Data cleaning research has evolved along several complementary directions, each addressing different aspects of the problem. We position our work by surveying these traditions and identifying gaps that motivate our hybrid approach. In the following subsections, we consider different data cleaning approaches starting from traditional methods and moving towards recent LLM-assisted approaches.

Classical tabular data cleaning has relied heavily on explicit constraints, external knowledge, and probabilistic inference [27]. Early work surveyed the problem space and established cleaning as a core data-management task. Later systems operationalized different forms of external or structured knowledge: `KATARA` uses knowledge bases and crowdsourcing to interpret and repair table values [8], `HoloClean` frames cleaning as probabilistic inference by combining integrity constraints with statistical signals to estimate the most likely clean database [28], and `Llunatic` provides a general chase-based framework for data cleaning under integrity constraints and related repair settings [11]. `Auto-Detect` takes a different direction by leveraging statistics from large corpora of clean tables to detect anomalies without requiring manually specified constraints [15]. Recent work further questions some common assumptions in constraint-based repair, showing that minimum-change repairs are not always the most appropriate cleaning outcomes [6]. Broader deployment-oriented evaluations also show that automatic data repair methods vary substantially across error rates, error types, and downstream tasks [25]. These systems demonstrate the value of grounding cleaning decisions in structured evidence. However, they generally do not treat validated historical corrections as a separate, append-only memory source that grows through iterative human feedback and can be retrieved alongside stable domain knowledge.

2.2 Example-Driven and Semi-Supervised Cleaning

A complementary line of work reduces the burden of manually specifying rules by learning from limited user supervision. `Raha` formulates error detection as a configuration-free classification problem that combines signals from multiple pre-existing error detection strategies without requiring task-specific rule authoring [23], while more recent work has explored few-shot and meta-learning formulations for tabular error detection [2]. `ED2` studies active learning strategies for error detection, prioritizing labeling effort to maximize detection performance [24], and `Baran` performs semi-supervised error correction by learning

from a small number of example repairs using a unified contextual representation [22]. Recent work has also examined how to accelerate `Raha` and `Baran` through task and data parallelism, highlighting both their practical relevance and the computational challenges of scaling such cleaning pipelines [4]. This literature is important for `AstraClean-RAG` because it shows that small amounts of validated human feedback can be generalized effectively. Our work differs in that it preserves validated corrections as an explicit case-like memory and combines them with a separate rule-like knowledge source, rather than using supervision solely to train a detection or correction model.

2.3 LLM-Assisted and Retrieval-Grounded Cleaning

Recent work has begun to explore LLM-assisted cleaning and retrieval-grounded repair. `RetClean` retrieves similar tuples from data lakes to support LLM-based cleaning [10], `IterClean` uses an iterative detector–verifier–repairer workflow [26], and `Cocoon` decomposes data cleaning into semantically structured sub-tasks for LLM orchestration [29]. Domain-specific studies also suggest that LLMs can be effective for cleaning structured personal contact data [12]. Other recent work has examined cost–quality trade-offs in LLM-based cleaning [13], and broader surveys now place such approaches within the emerging landscape of LLMs for table processing [21]. Compared with these systems, `AstraClean-RAG` does not rely on a large pre-existing clean reference lake as its only external source. Instead, it separates stable domain knowledge from evolving correction history and treats disagreements between them as first-class signals for review.

2.4 RBR/CBR as a Conceptual Lens for Dual-Source Cleaning

To clarify the architecture, we use rule-based reasoning (RBR) and case-based reasoning (CBR) as a conceptual lens rather than claim a full classical expert system. In classical CBR, new problems are handled through the cycle of retrieving, reusing, revising, and retaining prior cases [1]. RBR, by contrast, applies explicitly represented general knowledge, such as rules or constraints, to new instances [9, 20, 18]. The two paradigms are widely treated as complementary in hybrid reasoning systems, as rules support consistency and explicit normative knowledge, whereas cases support contextual reuse and adaptation [18]. In `AstraClean-RAG`, the DKB is therefore *rule-like* rather than 'retrieved text', and the Correction Log is *case-like* rather than 'history'. The novelty of our design lies in keeping these sources separate, mediating between them, and allowing validated corrections to accumulate as future cases.

Human-in-the-loop design is central here not only as a safeguard against incorrect autonomous repair, but also as the mechanism by which the case base evolves over time through validated corrections. Our work differs by positioning the LLM as a mediator that integrates evidence from multiple sources and presents it to the user, rather than making autonomous repair decisions. In this sense, human participation is not only a safeguard, but also a mechanism for

maintaining and extending the case base over time through validated corrections.

3 System Design

3.1 Design Goals and Architecture Overview

`AstraClean-RAG` is designed around four goals: grounded suggestions, cumulative learning, transparent conflict handling, and human authority. Grounded suggestions ensure that outputs are supported by retrieved evidence rather than parametric knowledge alone. Cumulative learning allows validated corrections to be reused when similar error patterns recur. Transparent conflict handling makes disagreements between evidence sources visible instead of resolving them silently. Human authority ensures that final cleaning decisions remain with the reviewer.

To support these goals, the system combines two complementary knowledge sources within a unified retrieval and mediation pipeline. The first is a *Domain Knowledge Base* (DKB), which stores stable domain rules such as format conventions, valid ranges, and canonical representations. The second is a *Correction Log* (CL), which stores human-verified corrections made during previous cleaning sessions together with their context and provenance.

Figure 1 shows the overall workflow. Given a user-selected target column and lightweight cleaning instructions, the system generates a repair suggestion for each cell value by retrieving evidence from both the DKB and the CL and passing it to an LLM-based mediation layer. The human reviewer then determines whether each suggested repair represents a true error and whether to accept, modify, or reject it.

3.2 Core Data Structures

`AstraClean-RAG` relies on two complementary knowledge sources: a Domain Knowledge Base and a Correction Log. The DKB is a static, rule-based repository that captures stable domain constraints and conventions, while the CL is an append-only repository that stores human-verified corrections together with contextual metadata and provenance. The DKB stores stable domain rules for each dataset. Each entry contains a rule description, the target column, the rule type, and a short textual summary used for retrieval. The rule types include format constraints, valid ranges, canonical representations, and domain-specific constraints. The DKB is initialized once per dataset and remains fixed during cleaning.

The CL stores human-verified corrections collected during cleaning. Each entry records the dirty value, the verified clean value, the target column, the surrounding row context, and metadata such as reviewer identity and timestamp. The CL grows monotonically, and entries are never deleted or modified. This design preserves a complete audit trail and supports both case-based reasoning,

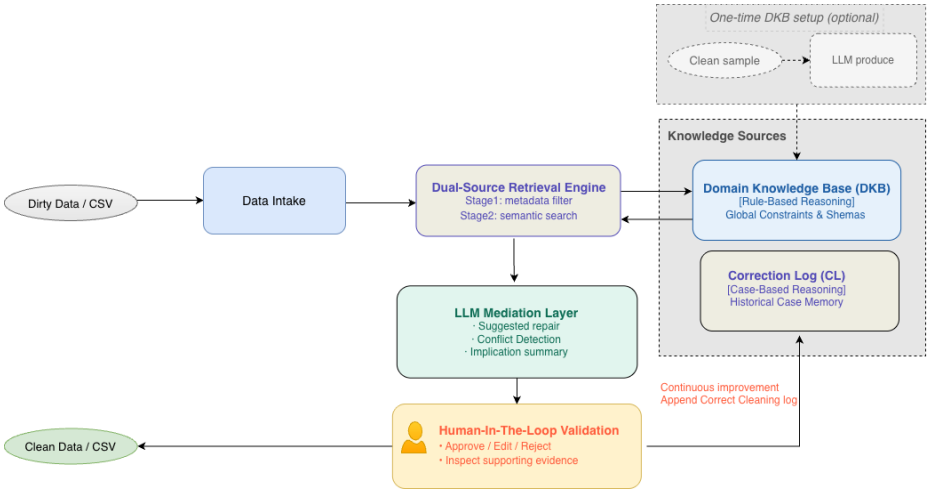


Fig. 1. High-level architecture and workflow of `AstraClean-RAG`; the system combines stable domain knowledge (DKB) and evolving correction history (CL) through dual-source retrieval and LLM-based mediation

through retrieval of similar past corrections, and interpretability, through traceable evidence. To support semantic search, both DKB and CL entries are embedded and stored in separate vector indices within a persistent vector database such as `Qdrant`.

3.3 Domain Knowledge Base Construction

In many practical settings, formal domain documentation is incomplete or unavailable. `AstraClean-RAG` therefore constructs the DKB through a lightweight semi-automated process based on clean data samples.

Given a dataset $\mathcal{D}$, the system first draws a stratified sample $\mathcal{S} \subseteq \mathcal{D}$:

$$|\mathcal{S}| = \max(0.1 \cdot |\mathcal{D}|, 100 \text{ rows}) \tag{1}$$

This sample is intended to preserve the distribution of major categorical attributes so that the generated rules are not dominated by overrepresented cases. The sampled rows are then provided to an LLM, which proposes candidate rules for each column. These rules may describe expected value formats, unit conventions, valid ranges, canonical forms, and simple domain-specific restrictions. The generated rules are treated strictly as candidates rather than final knowledge. Before indexing, a human reviewer inspects each candidate rule together with its target column and supporting sample values. The reviewer may accept, revise, merge, delete, or manually add rules. Poor rules, including rules that are overly general, unsupported by the sample, or inconsistent with known domain conventions, are revised or removed. If candidate rules are contradictory, they

are not committed automatically; the reviewer either resolves the contradiction by rewriting the rule set or excludes the conflicting rules when the available evidence is insufficient. Thus, the DKB construction process is semi-automated rather than fully automatic: the LLM reduces the authoring burden, while the reviewer remains responsible for validating the rules that enter the knowledge base.

3.4 Dual-Source Retrieval and LLM-Based Mediation

For each candidate cell under review, `AstraClean-RAG` retrieves evidence from both the DKB and the CL. A key design challenge is avoiding cross-column noise during retrieval. For example, a value such as '12.0' in a volume column should not retrieve semantically similar but contextually irrelevant entries from a temperature column, where the same number would represent a different type of measurement. To reduce such mismatches, retrieval is carried out in two stages. The first stage is metadata filtering. Before vector similarity is computed, the search space is restricted to entries associated with the same target column. This removes most irrelevant matches before semantic retrieval begins. The second stage is vector search within the filtered candidates. The query embedding is constructed from the target column, the dirty value, and row context:

$$\text{query} = \text{embed}(\text{“column: } c \mid \text{dirty: } v_{\text{dirty}} \mid \text{context: } x\text{”}) \tag{2}$$

where v_{dirty} is the anomalous cell value and x denotes relevant row-level context from the same record. The system retrieves the top-k results from each source independently. By default, $k = 3$ for the DKB and $k = 3$ for the CL. This value was selected to balance retrieval coverage and prompt length.

The retrieved evidence is then passed to an LLM-based mediation layer. Rather than making a fully autonomous cleaning decision, the LLM compares the retrieved DKB and CL evidence and classifies the case as either `aligned` or `conflicting`. A case is treated as `aligned` when both sources support the same target value or compatible normalization rationale. A case is treated as `conflicting` when the DKB and CL suggest different target values, incompatible formats, or different normalization policies. In aligned cases, the system generates a grounded repair suggestion with a short justification and supporting evidence identifiers. In conflicting cases, the system does not choose one source automatically; instead, it produces a structured conflict summary describing the DKB interpretation, the CL pattern, and the nature of the disagreement, leaving the final decision to the human reviewer. The mediation procedure is summarized in Algorithm 1.

3.5 Human Review and Feedback Loop

`AstraClean-RAG` is designed as a decision-support system in which final authority remains with the human reviewer. The review interface presents the cell value

Algorithm 1 LLM-Based Mediation

```
procedure MEDIATE(results_dkb, results_cl)
    context_dkb ← format_evidence(results_dkb)
    context_cl ← format_evidence(results_cl)
    output ← LLM(mediation instruction, {context_dkb, context_cl})
    if output.mode = aligned then
        return format_suggestion(output)
    else
        return format_conflict_summary(output)
    end if
end procedure
```

under review, its target column, the surrounding row context, the system output, and the retrieved evidence from both the DKB and the CL.

The human reviewer may then approve the suggestion, modify it before approval, or reject it and enter a different clean value manually. Each validated decision is then appended to the CL together with its context, metadata, and provenance. The recorded provenance includes which DKB rules were retrieved, which historical corrections were shown, what action the reviewer took, and when the decision was made. This feedback loop allows the CL to grow over time and makes repeated correction patterns easier to reuse in later sessions. At the same time, it preserves an auditable record of how decisions were supported and finalized.

3.6 Proof-of-Concept Implementation

We implemented a proof-of-concept web interface for `AstraClean-RAG`, shown in Fig. 2. In the data cleaning view, users can upload a CSV file, select a target column, provide lightweight cleaning instructions, and review flagged anomalies one by one. For each candidate cell under review, the interface displays the suggested repair, the retrieved evidence, and, when applicable, a structured conflict summary. More importantly, it presents the retrieved evidence alongside each suggestion so that reviewers can inspect the basis of the system output rather than treating it as a black box.

Fig. 3 illustrates the data index management view. This module is used to maintain the two main evidence sources in the system: domain knowledge and correction history. Through this interface, users can create, update, and delete `domain_kb` and `history_log` entries. In addition, the module supports domain knowledge construction from clean data samples. Candidate rules are generated by an LLM, presented to the user for optional review and editing, and then saved in JSONL format and added to the retrieval index for subsequent cleaning.

The backend uses `FastAPI` (Python), and the frontend uses `React` (JavaScript). For retrieval, we use the `sentence-transformers/all-MiniLM-L6-v2` embedding model to generate dense vector representations, and `Qdrant` as the vector database for indexing and similarity search. We use GPT-3.5 as the underlying

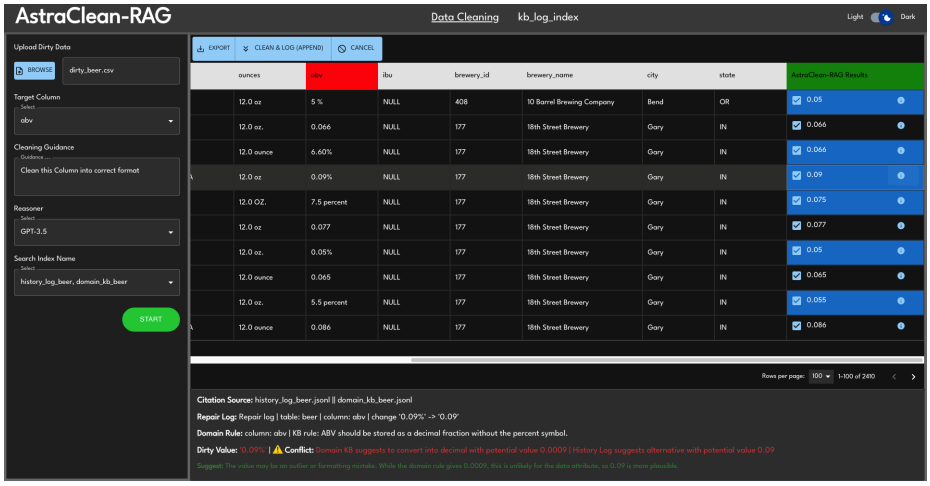


Fig. 2. Proof-of-concept interface of `AstraClean-RAG`, showing a suggested repair, retrieved evidence from DKB rules and CL corrections, and a conflict indicator for cases where the two sources suggest different resolutions

language model. These implementation choices reflect the objective of this study, which is to evaluate the effect of dual-source retrieval and evidence integration rather than model-specific improvements. Therefore, we rely on lightweight and widely used components instead of optimizing for the strongest possible model performance.

4 Empirical Evaluation

4.1 Datasets

We evaluate `AstraClean-RAG` on three datasets, summarized in Table 1, selected to cover complementary data cleaning scenarios. The dataset *Beers* focuses primarily on numeric normalization and unit conversion, including inconsistent representations of ounces, precision variations in alcohol content, and missing values. *Hospital* emphasizes textual and semi-structured cleaning problems, including typographical errors, abbreviation inconsistency, entity normalization, malformed phone numbers, and identifier format violations. *Tax* provides a larger and more rule-intensive setting, where cleaning decisions depend on stricter structural constraints such as zip code formatting and numeric standardization. For each dataset, the domain knowledge base is initialized from a clean sample with lightweight human verification. The correction log is initialized as empty and populated progressively during evaluation. Although these datasets are commonly used in data cleaning research, recent work has argued that benchmark error models may not fully reflect realistic tabular data errors [17]. We therefore interpret the evaluation as a controlled benchmark study

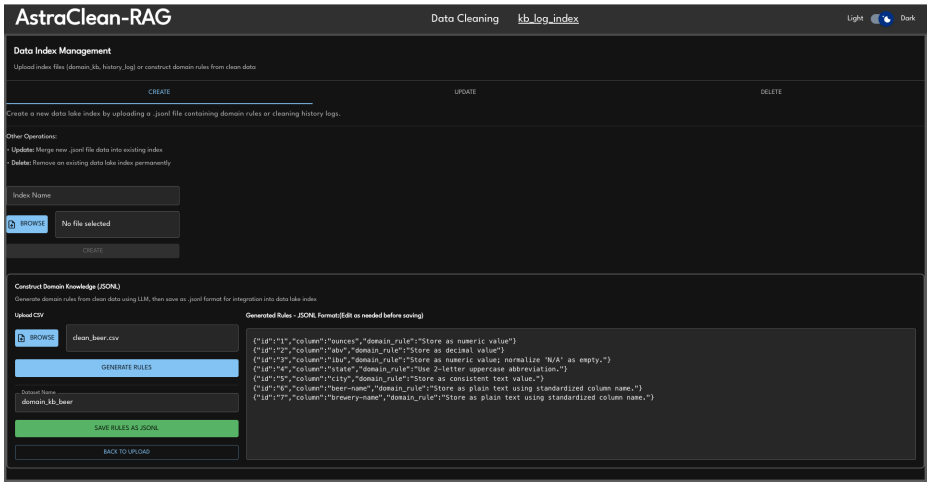


Fig. 3. Data index management interface of `AstraClean-RAG`, allowing users to manage domain knowledge and correction logs and to construct domain rules from clean data using an LLM with optional human review before indexing

of dual-source retrieval behavior rather than as a definitive assessment of performance in all real-world cleaning settings.

Table 1. Datasets used in the evaluation, adapted from benchmark settings in `Baran` [22]

Dataset	Rows	Cols	Dirty Cells	Error Types
Beers	2,410	9	∼270	Units, precision, missing values
Hospital	1,000	20	∼330	Typos, abbreviations, malformed fields
Tax	2,000	15	∼800	ZIP format, rate precision, normalization

4.2 Baselines and Main Results

We evaluate `AstraClean-RAG` against three baselines: LLM-only (zero-shot), which uses GPT-3.5 without any retrieval; DKB-only, which retrieves from the domain knowledge base alone; and CL-only, which retrieves from the correction log alone. The main results are reported on two representative dirty columns from each dataset, chosen to reflect different cleaning challenges within the same domain. For *Beers*, we evaluate on *ounces* and *abv*; for *Hospital*, on *city* and *measure_name*; and for *Tax*, on *zip* and *rate*. These columns were selected because they capture recurring but distinct forms of noise, including unit inconsistency, numeric formatting, typographical errors, and rule-intensive normalization.

For both CL-only and `AstraClean-RAG`, the correction log is initialized as empty and populated progressively from previously validated corrections in the training split. For the main results, each method uses 20 historical corrections per selected column. At each evaluation stage, both methods access the same amount of historical correction data, ensuring a fair comparison without exposure to future corrections. Precision (P) measures the fraction of suggested repairs that are correct; Recall (R) measures the fraction of true errors for which a correct repair was suggested; F1 is their harmonic mean.

Table 2. Main results on two representative dirty columns from each dataset, evaluated using precision (P), recall (R), and F1-score

	Beers			**Hospital**			**Tax**		
Method	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**
LLM-only	0.75	0.82	0.78	0.83	0.87	0.85	0.71	0.79	0.75
DKB-only	0.82	0.80	0.81	0.88	0.85	0.86	0.89	0.84	0.86
CL-only	0.86	0.89	0.87	0.86	0.88	0.87	0.81	0.85	0.83
`AstraClean-RAG`	0.89	0.91	0.90	0.88	0.89	0.88	0.91	0.86	0.88

Table 2 summarizes the results. Across all datasets, `AstraClean-RAG` achieves the highest overall F1-score, while exhibiting a more balanced precision–recall trade-off than the single-source baselines. On the *Beers* dataset, improvements are most evident in columns such as *ounces* and *abv*. The *ounces* column contains recurring unit inconsistencies (e.g., "12.0 oz", "12.0 oz.", "355 ml", "0.355L"), where similar correction patterns appear repeatedly. This favors case-based reuse and helps explain the relatively strong performance of CL-only. In contrast, the *abv* column involves format normalization (e.g., "0.05" vs "0.05%"), where domain constraints provide stronger guidance.

On the *Hospital* dataset, the gains are more moderate due to the diversity and irregularity of errors. For example, the *city* column contains spelling variations (e.g., "birminghxm", "sheffxeld"), while *measure_ name* includes longer and less structured textual noise (e.g., "caxxed", "bxockers", "hospitax"). Such errors are less repetitive and therefore harder to capture through either rules or historical reuse alone, resulting in smaller but still consistent improvements.

On the *Tax* dataset, DKB-only achieves relatively high precision, reflecting the importance of explicit structural constraints in rule-intensive settings. For instance, the *zip* column requires consistent formatting with leading zeros (e.g., "01907" vs "1907"), and the *rate* column exhibits precision inconsistencies (e.g., "5", "5.3", "6.84", "1.9519792"). In this setting, domain rules provide strong guidance, while the correction log contributes additional coverage for recurring edge cases.

Overall, these results indicate that the two knowledge sources provide complementary benefits: domain knowledge supports precision through stable con-

straints, while historical corrections improve coverage by capturing recurring patterns.

4.3 Longitudinal Analysis

Unlike the main results in Table 2, which are aggregated over two representative columns, this analysis focuses only on the *ounces* column of the *Beers* dataset. We simulate system performance as the correction log grows over time under the full system configuration, where both the domain knowledge base and the correction log are used.

Table 3. Longitudinal performance on the *ounces* column of the *Beers* dataset

Number of corrections in CL	F1
0	0.78
10	0.85
20	0.88
50	0.89

The results show that performance improves rapidly in the early stage, with F1 increasing from 0.78 to 0.88 within the first 20 corrections. This indicates that frequent error patterns are quickly captured and reused. Further gains beyond 20 corrections are smaller, suggesting that most common patterns have already been covered and that additional corrections mainly address less frequent cases. The results suggest that the correction log is most beneficial in the early stage, while the DKB provides stable support before sufficient historical corrections accumulate.

4.4 Conflict Analysis

We analyze cases where the domain knowledge base and the correction log suggest different repairs. Such conflicts arise when global constraints and historical corrections provide inconsistent signals. For example, in the *rate* column of the *Tax* dataset, domain rules may favor a fixed numeric precision, while historical corrections reflect varying but previously accepted levels of precision. Similarly, in the *ounces* column of the Beers dataset, domain rules may favor a canonical unit representation, while a particular historical correction may preserve a previously accepted converted form. In both cases, the disagreement does not necessarily indicate an error in either source. Rather, it reflects different but reasonable normalization choices. Instead of resolving these disagreements automatically, `AstraClean-RAG` presents both interpretations to the reviewer and leaves the final decision to human judgment.

5 Discussion

5.1 Practical Implications

In real-world data cleaning workflows, organizations often lack large-scale clean reference resources, while knowledge about how to resolve recurring errors remains scattered across people, documents, and past decisions. `AstraClean-RAG` explores one possible way to address this by externalizing two complementary forms of knowledge: stable domain conventions in a Domain Knowledge Base and validated historical corrections in a Correction Log. By representing these as explicit external knowledge sources rather than leaving them implicit in model behavior, the framework suggests a possible direction for retrieval-grounded LLM-assisted cleaning in settings where a large pre-cleaned reference repository is unavailable, but limited clean examples, domain documentation, or expert-reviewed samples can still be used to initialize lightweight domain knowledge. This also makes the basis of repair suggestions more explicit and easier to review. This may be particularly useful in settings where important cleaning knowledge is internal to the organization, such as private conventions, validated historical decisions, or domain-specific business logic.

The dual-source design also points to a different way of using human effort in the cleaning process. Instead of requiring experts to specify all rules upfront or repeatedly resolve similar issues manually, the framework allows knowledge support to develop incrementally through reviewed rules and validated corrections. Over time, this may help shift part of the human role from repeated manual cleaning toward knowledge curation, while preserving reviewer authority over final decisions. By surfacing the specific domain rule or historical correction behind each suggestion, the framework may also contribute to a more transparent and auditable cleaning workflow over time.

5.2 Limitations

This work has several limitations. First, the effectiveness of `AstraClean-RAG` depends on the quality and coverage of its two knowledge sources. The system is less effective when both the DKB and the CL provide weak or noisy signals, especially for errors that are neither covered by domain rules nor sufficiently similar to prior corrections. Moreover, the current DKB construction process relies on limited clean examples, domain documentation, or expert-reviewed samples. Therefore, `AstraClean-RAG` should not be interpreted as requiring no clean information at all; rather, it reduces reliance on a large pre-existing clean reference repository by combining lightweight, inspectable domain knowledge with an evolving correction log.

Second, the empirical evaluation is limited in scope. Although we evaluate the framework on multiple datasets, the main results are reported on selected representative columns rather than all dirty attributes, and the benchmark datasets are relatively small. These datasets also may not fully capture realistic tabular

errors. Future work should therefore evaluate the framework on larger datasets, a fuller range of dirty columns, and more realistic error models [17].

Third, the current study does not directly evaluate reviewer interaction or robustness across model backbones. We do not include a user study of reviewer decision time, cognitive effort, or the effect of structured conflict summaries compared with raw LLM suggestions. In addition, all experiments use GPT-3.5 as the underlying language model. While newer models may provide stronger reasoning capabilities, our objective is to evaluate the effect of dual-source retrieval rather than model-specific improvements. We reduce output variability by using a fixed model configuration and deterministic decoding where supported, with temperature set to 0; however, this does not guarantee fully identical outputs across runs. Future work should examine whether stronger LLMs reduce the marginal benefit of retrieval, or whether explicit DKB and CL evidence continues to improve grounding, traceability, and conflict visibility.

6 Conclusion

We presented `AstraClean-RAG`, a human-in-the-loop framework for tabular data cleaning that combines a fixed DKB and an append-only CL through dual-source retrieval and LLM mediation. By integrating rule-based and case-based reasoning, the framework is designed to balance consistency with adaptability in practical data cleaning scenarios. The empirical results indicate that combining domain knowledge with accumulated correction history can improve cleaning performance over single-source approaches on the evaluated tasks. In addition, the framework supports incremental learning from human feedback and treats conflicts between knowledge sources as signals to be exposed rather than automatically resolved. While the current evaluation focuses on selected columns and a single LLM backbone, the results suggest that structuring human–LLM collaboration around explicit evidence and complementary knowledge sources is a promising direction for knowledge-intensive data management tasks.

Acknowledgments. This work has been supported by FAST, the Finnish Software Engineering Doctoral Research Network, funded by the Ministry of Education and Culture, Finland.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Aamodt, A., Plaza, E.: Case-based reasoning: Foundational issues, methodological variations, and system approaches. AI Communications (1994)
2. Abdelaal, M., Ktitarev, T., Städtler, D., Schöning, H.: SAGED: Few-shot meta learning for tabular data error detection. In: Proceedings of the 27th International Conference on Extending Database Technology (EDBT 2024). pp. 386–398. OpenProceedings.org (2024). https://doi.org/10.48786/EDBT.2024.34

3. Abedjan, Z., Golab, L., Naumann, F.: Profiling relational data: a survey. The VLDB Journal **24**(4), 557–581 (Aug 2015). https://doi.org/10.1007/s00778-015-0389-y, https://doi.org/10.1007/s00778-015-0389-y
4. Ahmadi, F., Mandirali, Y., Abedjan, Z.: Accelerating the data cleaning systems Raha and Baran through task and data parallelism. Proceedings of the VLDB Endowment. ISSN **2150**, 8097 (2024)
5. Chen, B., Taipalus, T.: AstraClean-RAG: Supporting human-LLM collaboration in tabular data cleaning through dual-source retrieval. In: Proceedings of the 2026 ACM CHI Conference on Human Factors in Computing Systems (CHI'26), Co-Data: Cultivating Effective Human-LLM Collaboration for Collaborative Data Processing. ACM (2026), in press
6. Chen, J., Sun, Y., Song, S., Zhang, H., Yuan, X.: Minimum Change ≠ Best Cleaning: Parallel and incremental error detection under integrity constraints. Proc. ACM Manag. Data **3**(4) (Sep 2025). https://doi.org/10.1145/3749174, https://doi.org/10.1145/3749174
7. Chu, X., Ilyas, I.F., Krishnan, S., Wang, J.: Data cleaning: Overview and emerging challenges. In: Proceedings of the 2016 international conference on management of data. pp. 2201–2206 (2016)
8. Chu, X., Ouzzani, M., Morcos, J., Ilyas, I.F., Papotti, P., Tang, N., Ye, Y.: KATARA: Reliable data cleaning with knowledge bases and crowdsourcing. Proceedings of the VLDB Endowment **8**(12), 1952–1955 (2015). https://doi.org/10.14778/2824032.2824109
9. Dutta, S., Bonissone, P.P.: Integrating case-and rule-based reasoning. International Journal of Approximate Reasoning **8**(3), 163–203 (1993)
10. Eltabakh, M.Y., Naeem, Z.A., Ahmad, M.S., Ouzzani, M., Tang, N.: RetClean: Retrieval-based data cleaning using LLMs and data lakes. Proceedings of the VLDB Endowment **17**(12), 4421–4424 (2024). https://doi.org/10.14778/3685800.3685890
11. Geerts, F., Mecca, G., Papotti, P., Santoro, D.: Cleaning data with Llunatic. The VLDB Journal **29**(4), 867–892 (2020)
12. Glock, A.C., Dominka-Kiss, C., Korom, P., Ehrlinger, L.: Detecting and cleaning errors in personal contact information with large language models. In: Proceedings of the 2nd International Workshop on Data-driven AI (DATAI 2025) at VLDB 2025. VLDB Endowment (2025), https://www.vldb.org/2025/Workshops/VLDB-Workshops-2025/DATAI/DATAI25_12.pdf
13. Hu, T., Wang, J., Pu, W., Li, J., Gu, R., Bi, X., Yin, H., Wang, Y.P.: A multi-objective optimization framework for data cleaning using large language models. Big Data Mining and Analytics (2025). https://doi.org/10.26599/BDMA.2025.9020074
14. Huang, L., Yu, W., Ma, W., et al.: A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. arXiv preprint arXiv:2311.05232 (2023), https://arxiv.org/abs/2311.05232
15. Huang, Z., He, Y.: Auto-Detect: Data-driven error detection in tables. In: Proceedings of the 2018 International Conference on Management of Data. pp. 1377–1392. ACM (2018). https://doi.org/10.1145/3183713.3196889
16. Jäger, S., Biessmann, F.: From data imputation to data cleaning — automated cleaning of tabular data improves downstream predictive performance. In: Proceedings of The 27th International Conference on Artificial Intelligence and Statistics. Proceedings of Machine Learning Research, vol. 238, pp. 3394–3402. PMLR (2024), https://proceedings.mlr.press/v238/jager24a.html

17. Jung, P., Jäger, S., Chandler, N., Biessmann, F.: Towards realistic error models for tabular data. J. Data and Information Quality **17**(4) (Dec 2025). https://doi.org/10.1145/3774914, https://doi.org/10.1145/3774914
18. Kaoura, G., Kovas, K., Boutsinas, B., Hatzilygeroudis, I.: A new hybrid reasoning model based on rules, cases and processes: application to care of individuals facing autism spectrum disorders. Knowledge and Information Systems **67**, 371–401 (2025). https://doi.org/10.1007/s10115-024-02228-x
19. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in neural information processing systems **33**, 9459–9474 (2020)
20. Liao, S.H.: Expert system methodologies and applications—a decade review from 1995 to 2004. Expert systems with applications **28**(1), 93–103 (2005)
21. Lu, W., Zhang, J., Fan, J., Fu, Z., Chen, Y., Du, X.: Large language model for table processing: a survey. Frontiers of Computer Science **19**, 192350 (2025). https://doi.org/10.1007/s11704-024-40763-6
22. Mahdavi, M., Abedjan, Z.: Baran: Effective error correction via a unified context representation and transfer learning. Proceedings of the VLDB Endowment **13**(12), 1948–1961 (2020). https://doi.org/10.14778/3407790.3407801
23. Mahdavi, M., Abedjan, Z., Fernandez, R.C., Madden, S., Ouzzani, M., Stonebraker, M., Tang, N.: Raha: A configuration-free error detection system. In: Proceedings of the 2019 International Conference on Management of Data. pp. 865–882. ACM (2019). https://doi.org/10.1145/3299869.3324956
24. Neutatz, F., Mahdavi, M., Abedjan, Z.: ED2: A case for active learning in error detection. In: Proceedings of the 28th ACM International Conference on Information and Knowledge Management. pp. 2249–2252. ACM (2019). https://doi.org/10.1145/3357384.3358129
25. Ni, W., Miao, X., Zhao, X., Wu, Y., Liang, S., Yin, J.: Automatic data repair: Are we ready to deploy? Proc. VLDB Endow. **17**(10), 2617–2630 (Jun 2024). https://doi.org/10.14778/3675034.3675051, https://doi.org/10.14778/3675034.3675051
26. Ni, W., Zhang, K., Miao, X., Zhao, X., Wu, Y., Yin, J.: IterClean: An iterative data cleaning framework with large language models. In: Proceedings of the ACM Turing Award Celebration Conference - China 2024. ACM (2024). https://doi.org/10.1145/3674399.3674436
27. Rahm, E., Do, H.H.: Data cleaning: Problems and current approaches. IEEE Data Engineering Bulletin **23**(4), 3–13 (2000)
28. Rekatsinas, T., Chu, X., Ilyas, I.F., Ré, C.: HoloClean: Holistic data repairs with probabilistic inference. Proceedings of the VLDB Endowment **10**(11), 1190–1201 (2017). https://doi.org/10.14778/3137628.3137631
29. Zhang, S., Huang, Z., Wu, E.: Data cleaning using large language models. In: 2025 IEEE 41st International Conference on Data Engineering Workshops (ICDEW). pp. 28–32. IEEE (2025). https://doi.org/10.1109/ICDEW67478.2025.00008